

Concurrency and Distributed Systems

Week 7 – Part 1: CompletableFuture Pipelines

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 1 Roadmap

- 1 From Parallelism to Async Composition
- 2 CompletableFuture Basics
- 3 Demo

Week 6 vs Week 7

Week 6 question

Can one CPU-bound computation be decomposed into useful parallel work?

Week 7 question

Can several operations be composed even when they complete later, fail, or time out?

Important distinction

Async code is not automatically faster. It is a way to model completion, failure, and waiting.

What a Future Represents

- A value that may become available later.
- A failure that may be reported later.
- A continuation point for the next step in a pipeline.
- A boundary where the caller decides whether to wait, compose, or return another future.

A Minimal Pipeline

```
1 CompletableFuture<String> future =  
2   CompletableFuture  
3     .supplyAsync(() -> "profile:" + requestId)  
4     .thenApply(String::toUpperCase);  
5  
6 String result = future.join();
```

- `supplyAsync`: starts work that produces a value.
- `thenApply`: transforms the successful value.
- `join`: waits at the edge of the program.

Pipeline Shape



Executor Choice

```
1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
2     CompletableFuture<String> future =  
3         CompletableFuture.supplyAsync(this::callService, executor);  
4     return future.join();  
5 }
```

- Prefer explicit executors in teaching demos.
- The default async executor is the common pool.
- Use the executor that matches the workload and ownership boundary.

Run

```
.\run-demo.ps1 01
```

- Locate Demo01_CompletableFuturePipeline.
- Identify the asynchronous source step.
- Identify the transformation step.
- Identify where the program waits for completion.

Key Takeaway

A good async pipeline is explicit

It shows where work starts, how values transform, where errors are handled, and where waiting happens.